PayShield

Advanced Digital Banking System with Integrated Fraud Detection

System Design & Software Engineering Capstone Report

Team Composition

Dhanvin Vadlamudi	2401010150	*Team Lead & Architect*
Nipun Patlori	2401010323	
Tejaswini Palwai	2401010314	
Meka Chaitanya Sai	2401010275	
Killi Akshith Kumar	2401010230	

April 27, 2026

Department of Software Engineering
Rishihood University

Contents

1 Introduction **2**

2 System Architecture & Design **2**
2.1 Presentation Layer (Frontend) 2
2.2 Business Logic Layer (Backend) 2
2.3 Data Access Layer (Database) 2

3 Advanced Design Patterns (GoF) **2**
3.1 Singleton Pattern (Creational) 2
3.2 Factory Pattern (Creational) 3
3.3 Strategy Pattern (Behavioral) 3
3.4 Observer Pattern (Behavioral) 3
3.5 Command Pattern (Behavioral) 3

4 SOLID & OOP Integration **3**
4.1 The SOLID Paradigm 3
4.2 Object-Oriented Excellence 3

5 Functional Requirements & System Capabilities **4**
5.1 User Capability Modules 4
5.2 Administrative Oversight 4

6 Data Modeling & Persistence **4**

7 SDLC Methodology **4**

8 Conclusion & Future Scope **4**

1 Introduction

PayShield is a sophisticated digital banking ecosystem developed to demonstrate the synergy between classical Software Engineering principles and modern web scale-out architectures. Unlike conventional banking prototypes, PayShield is engineered around a "Security-First" paradigm, integrating a real-time Fraud Detection Engine directly into the transaction lifecycle.

The system addresses the critical problem of transactional security in the NoSQL era, proving that data integrity and complex business rules can be maintained without compromising the horizontal scalability provided by MongoDB.

2 System Architecture & Design

The system adheres to a strict **3-Tier Layered Architecture**, ensuring high modularity and loose coupling between components.

2.1 Presentation Layer (Frontend)

Developed using **React.js** and **TypeScript**, the frontend utilizes a custom CSS design system rather than generic utility frameworks. This ensures a unique, high-premium aesthetic tailored for financial transparency. State management is handled through React Context (AuthContext), ensuring secure JWT propagation across the application.

2.2 Business Logic Layer (Backend)

The core intelligence resides in the **Node.js/Express** server. This layer is entirely decoupled from both the UI and the Database, communicating via standardized RESTful endpoints. It implements the primary business rules including interest calculation, overdraft protection, and the fraud detection pipeline.

2.3 Data Access Layer (Database)

MongoDB Atlas provides the persistence layer. By utilizing a document-based model, PayShield handles the polymorphic nature of banking products (e.g., Savings vs. Checking accounts) without the rigid schema constraints of traditional SQL, while maintaining relationship integrity through ObjectId referencing.

—

3 Advanced Design Patterns (GoF)

PayShield implements five distinct Gang of Four (GoF) design patterns to ensure the codebase remains "Enterprise-Ready."

3.1 Singleton Pattern (Creational)

File: `src/server/config/database.ts`
The `DatabaseConnection` class ensures that only a single instance of the MongoDB client is instantiated. This prevents "Connection Pool Exhaustion" and ensures that the application maintains a stable heartbeat with the cloud cluster.

3.2 Factory Pattern (Creational)

File: `src/server/patterns/factory/AccountFactory.ts`
To handle the creation of diverse financial products, we implemented an `AccountFactory`. This abstracts the instantiation logic for `SavingsAccount` and `CheckingAccount`, allowing the system to scale to new account types (e.g., Business, Crypto) without modifying the creation logic in the controllers.

3.3 Strategy Pattern (Behavioral)

File: `src/server/fraud/FraudDetectionEngine.ts`
The fraud detection engine utilizes the Strategy pattern to make rules interchangeable. Each rule (High Value, Rapid Transaction, New Recipient) implements the `IFraudStrategy` interface. This enables the system to run multiple security audits polymorphically on every transaction.

3.4 Observer Pattern (Behavioral)

File: `src/server/patterns/observer/EventEmitter.ts`
When a transaction is flagged, the system uses an event-driven approach. The `FraudDetectionEngine` acts as a "Subject," emitting an event that the `FraudAlertObserver` listens for. This decouples the security analysis from the alert logging and reporting logic.

3.5 Command Pattern (Behavioral)

File: `src/server/patterns/command/TransferCommand.ts`
Every fund transfer is encapsulated as a `Command` object. This treats transactions as first-class citizens, providing an immutable history and enabling the future implementation of complex "Undo" (Reversal) operations with high data integrity.

—

4 SOLID & OOP Integration

4.1 The SOLID Paradigm

S - Single Responsibility: Every class and service (e.g., `AuthService`, `LoanService`) has a single, well-defined reason to change.

O - Open/Closed: The Fraud Engine is open for new detection strategies but closed for modification of the orchestration engine.

L - Liskov Substitution: Specialized accounts can be substituted for base accounts in the transaction logic without breaking system invariants.

I - Interface Segregation: We utilize lean, focused interfaces like `IUser` and `IAccount` rather than monolithic "fat" interfaces.

D - Dependency Inversion: Controllers depend on high-level Service abstractions rather than low-level database implementation details.

4.2 Object-Oriented Excellence

PayShield leverages **Inheritance** for hierarchical account modeling and **Encapsulation** to protect sensitive financial logic. **Abstraction** is utilized to hide the complexity of Mongoose aggregation pipelines behind simple, readable service methods.

—

5 Functional Requirements & System Capabilities

5.1 User Capability Modules

- **Identity & Access:** Secure multi-tenant authentication using JWT and Bcrypt hashing.
- **Product Management:** On-demand creation of Savings/Checking accounts with automated interest rates.
- **Transaction Engine:** Real-time fund transfers with atomic-like balance updates.
- **Credit Oversite:** Loan application submission with EMI projection and status tracking.

5.2 Administrative Oversight

- **Fraud Monitoring:** Real-time review of flagged transactions with severity scoring.
- **Loan Processing:** Administrative control over credit approval/rejection workflows.
- **Financial Audit:** Global oversight of system-wide liquidity and user accounts.

—

6 Data Modeling & Persistence

The database utilizes a **Referential NoSQL** approach. - **User Collection:** Stores hashed credentials and role-based metadata. - **Account Collection:** Links to User via `userId` reference, storing current balance and account type. - **Transaction Collection:** Stores immutable logs of money movement between account ObjectIds. - **FraudAlert Collection:** Links specific Transaction IDs to triggered security rules.

—

7 SDLC Methodology

PayShield was developed using the **Agile-Scrum** methodology. We utilized 6 rapid sprints:

- **Sprint 1:** Requirements Analysis & Data Modeling.
- **Sprint 2:** Core Backend & Authentication Infrastructure.
- **Sprint 3:** Design Pattern Implementation (Creational Behavioral).
- **Sprint 4:** Frontend UI Development & UX Prototyping.
- **Sprint 5:** Fraud Detection Integration Security Auditing.
- **Sprint 6:** Final Optimization & Multi-Platform Deployment.

—

8 Conclusion & Future Scope

PayShield demonstrates that modern web frameworks can achieve enterprise-grade stability when coupled with robust System Design patterns. By utilizing TypeScript for end-to-end type safety and GoF patterns for architectural flexibility, the system stands as a high-fidelity blueprint for secure digital banking.

Future Scope: The modular architecture allows for seamless integration of AI-driven fraud prediction models and the expansion into multi-currency cross-border payment rails.

PayShield

Secure. Smart. Reliable.

End of Project Report - 2026